

INF672 · MACHINE LEARNING · GUÍA DE LABORATORIO

GL09

Regresión lineal en dos caminos

A · con scikit-learn | B · con descenso de gradiente desde cero

ASIGNATURA	INF672 · Machine Learning	UNIDAD	3 · Regresión y flujo de trabajo del ML
TEMA	3.1 Regresión lineal y descenso de gradiente	GESTIÓN	2026-02
DOCENTE	M. Sc. Huáscar F. Gonzales G.	DURACIÓN	1 sesión de laboratorio
CARRERA	Ingeniería Informática · UATF	ENTORNO	Python 3 · Jupyter / Google Colab

IDEA CLAVE Una pregunta, dos respuestas

En esta práctica resolvemos **el mismo problema** —encontrar la recta que mejor relaciona las horas de estudio con la nota de un examen— de dos maneras. Primero con `LinearRegression` de scikit-learn, que da la solución exacta en dos líneas. Después programamos el **descenso de gradiente** con NumPy, para ver qué ocurre por dentro. Al final comprobaremos que ambos caminos llegan a los mismos parámetros hasta la séptima cifra decimal.

01 Objetivos

1.1 Objetivo general

Construir, entrenar y evaluar un modelo de regresión lineal simple por dos vías —la solución directa de scikit-learn y el descenso de gradiente implementado desde cero— y verificar que ambas conducen al mismo resultado.

1.2 Objetivos específicos

1. Aplicar la interfaz estándar de scikit-learn (**crear** → **fit** → **predict**) a un problema de regresión, incluida la división en conjuntos de entrenamiento y prueba.
2. Calcular e interpretar las métricas de regresión **MAE**, **MSE**, **RMSE** y **R²**, tanto con `sklearn.metrics` como programándolas a mano.
3. Formular la función de costo $J(w, b)$ y derivar sus gradientes con la regla de la cadena.
4. Implementar con NumPy la regla de actualización del descenso de gradiente y el ciclo de entrenamiento completo.

5. Diagnosticar la convergencia mediante la **curva de aprendizaje** y reconocer el efecto de la tasa de aprendizaje α y de las variables sin escalar.

02 Competencias a desarrollar

Tipo	Competencia
Conceptual	Explica la regresión lineal como un problema de optimización: elegir los parámetros que minimizan el error cuadrático medio.
Procedimental	Implementa y compara un modelo de biblioteca y un algoritmo propio sobre los mismos datos, con semillas que garantizan reproducibilidad.
Analítica	Lee una curva de aprendizaje, identifica mesetas, sobrepasos y divergencia, y propone ajustes de α o de escalado.
Actitudinal	Valida sus resultados contra una referencia conocida en lugar de aceptar ciegamente la salida de un modelo.

03 Prerrequisitos

- Operaciones vectorizadas con NumPy: `np.mean`, `np.sum`, `reshape`, *broadcasting* (GL02–GL03).
- Derivadas, derivadas parciales y regla de la cadena (GL04 y apunte de derivadas).
- Idea de conjunto de entrenamiento y de prueba, y uso básico de scikit-learn (GL07).
- Gráficos de dispersión y de líneas con Matplotlib.

04 Materiales y entorno

Recurso	Detalle
Lenguaje	Python 3.10 o superior
Bibliotecas	<code>numpy</code> , <code>matplotlib</code> , <code>scikit-learn</code>
Entorno	Jupyter Notebook / JupyterLab (Anaconda) o Google Colab
Datos	Generados en el propio cuaderno (60 estudiantes simulados); no requiere archivos externos

05 Marco teórico

5.1 El modelo de regresión lineal simple

La regresión lineal simple busca la recta que mejor describe la relación entre una variable de entrada x (característica) y una salida numérica y (objetivo). La predicción del modelo, que se lee «y gorro», es:

$$\hat{y} = w \cdot x + b \quad (1)$$

donde w es la **pendiente** (cuánto cambia $\hat{y}$ por cada unidad de x) y b el **intercepto** (el valor de $\hat{y}$ cuando $x = 0$). Entrenar el modelo es encontrar los valores de w y b a partir de los datos.

5.2 La función de costo

Para decidir qué recta es «mejor» necesitamos medir su error. Para cada muestra i el **residuo** es $\hat{y}^{(i)} - y^{(i)}$. Elevamos los residuos al cuadrado —para que los positivos y negativos no se cancelen y para penalizar más los errores grandes— y los promediamos sobre las m muestras:

$$J(w, b) = (1 / 2m) \cdot \sum_{i=1..m} (w \cdot x^{(i)} + b - y^{(i)})^2 \quad (2)$$

Es el error cuadrático medio (MSE) con un factor $1/2$ añadido por conveniencia: al derivar, el 2 que baja del exponente se cancela con él. El factor no cambia **dónde** está el mínimo, solo su valor.

DEFINICIÓN Entrenar = minimizar

Entrenar un modelo de regresión lineal significa encontrar el par (w, b) que hace $J(w, b)$ lo más pequeño posible. Todo lo que sigue son dos estrategias distintas para llegar a ese mínimo.

5.3 Dos caminos hacia el mínimo

Camino directo (analítico). Para la regresión lineal el mínimo de J puede calcularse con una fórmula cerrada, la solución de **mínimos cuadrados ordinarios**. En el caso simple se reduce a:

$$\begin{aligned} w &= \sum (x^{(i)} - \bar{x})(y^{(i)} - \bar{y}) / \sum (x^{(i)} - \bar{x})^2 \\ b &= \bar{y} - w \cdot \bar{x} \end{aligned} \quad (3)$$

Es lo que hace internamente `LinearRegression` de scikit-learn: no itera, resuelve el problema algebraicamente de una sola vez.

Camino iterativo (descenso de gradiente). Partimos de valores arbitrarios de w y b y damos pasos repetidos «cuesta abajo» sobre la superficie de J . La dirección de cada paso la da el **gradiente** —el vector de derivadas parciales— y su tamaño, la **tasa de aprendizaje** α :

$$\begin{aligned} w &\leftarrow w - \alpha \cdot \partial J / \partial w \\ b &\leftarrow b - \alpha \cdot \partial J / \partial b \end{aligned} \quad (4)$$

Este camino es más lento para la regresión lineal, pero es el que se generaliza: la regresión logística y las redes neuronales **no tienen** solución cerrada y se entrenan precisamente así.

5.4 Métricas de evaluación

Una vez entrenado, el modelo se evalúa sobre datos que **no vio** durante el entrenamiento. Usaremos cuatro métricas:

Métrica	Fórmula	Interpretación
MAE	$(1/m) \sum y - \hat{y} $	Error promedio, en las unidades de y (puntos de nota).
MSE	$(1/m) \sum (y - \hat{y})^2$	Promedio de errores al cuadrado; penaliza los errores grandes. Sus unidades son «puntos ² ».
RMSE	$\sqrt{\text{MSE}}$	Como el MSE, pero de vuelta en las unidades de y : comparable con el MAE.
R²	$1 - SS_{\text{res}} / SS_{\text{tot}}$	Fracción de la variabilidad de y explicada por el modelo. 1 = perfecto; 0 = no mejor que predecir siempre la media.

06 Preparación del entorno y de los datos

6.1 Importaciones y estilo de gráficos

Comenzamos importando NumPy y Matplotlib y fijando la paleta de colores del curso para que todos los gráficos sean coherentes.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

# Paleta del curso
INDIGO, PURPURA, CORAL, TINTA = '#4F46E5', '#7E22CE', '#F43F5E', '#111827'
plt.rcParams.update({'figure.figsize': (7, 4.2),
                    'axes.grid': True, 'grid.alpha': 0.3,
                    'axes.spines.top': False,
                    'axes.spines.right': False})
```

6.2 Un conjunto de datos con respuesta conocida

Simulamos las **horas de estudio** (0 a 10) de 60 estudiantes y la **nota** obtenida en un examen (0 a 100). Los datos se construyen a partir de una relación «verdadera» conocida más un ruido aleatorio gaussiano:

$$\text{nota} = 30 + 5 \cdot \text{horas} + \varepsilon, \quad \varepsilon \sim N(0, 5^2) \quad (5)$$

Trabajar con datos simulados tiene una gran ventaja didáctica: conocemos de antemano la respuesta correcta ($w = 5$, $b = 30$) y podemos juzgar qué tan bien la recupera cada método.

```
In [2]: rng = np.random.default_rng(42)          # semilla: resultados reproducibles
        m = 60                                  # número de muestras

        horas = rng.uniform(0, 10, m)
        nota = 30 + 5 * horas + rng.normal(0, 5, m)

        print('Primeras 5 muestras:')
        for h, n in zip(horas[:5], nota[:5]):
            print(f'  horas = {h:5.2f}  nota = {n:6.2f}')
```

```
Out[2]: Primeras 5 muestras:
        horas = 7.74  nota = 64.37
        horas = 4.39  nota = 56.79
        horas = 8.59  nota = 64.52
        horas = 6.97  nota = 63.19
        horas = 0.94  nota = 35.52
```

ADVERTENCIA La semilla importa

`default_rng(42)` fija el generador de números aleatorios. Sin ella, cada ejecución produciría datos distintos y tus resultados no coincidirían con los de esta guía ni con los de tus compañeros. Siempre que un experimento involucre azar, fija la semilla.

```
In [3]: plt.scatter(horas, nota, color=INDIGO, alpha=0.8)
        plt.xlabel('Horas de estudio'); plt.ylabel('Nota')
        plt.title('Datos: horas de estudio vs. nota')
        plt.show()
```

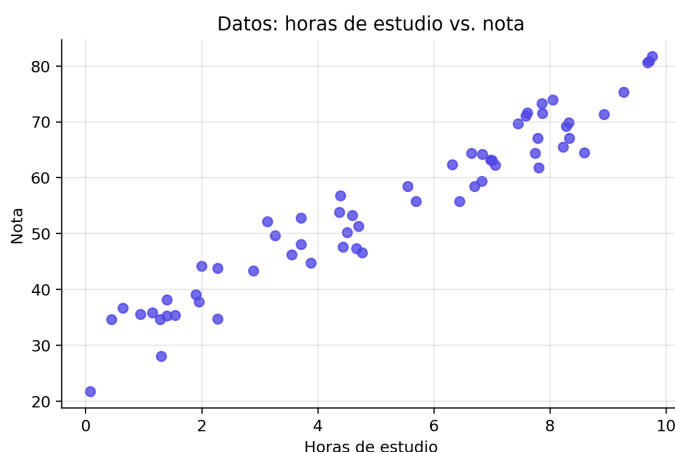


Figura 1. Diagrama de dispersión de los 60 estudiantes simulados.

Se aprecia una tendencia creciente con dispersión: más horas de estudio tienden a dar mejores notas, pero no de forma exacta. Una recta es un modelo razonable para estos datos.

07 Parte A — Regresión lineal con scikit-learn

scikit-learn organiza **todos** sus modelos con la misma interfaz de tres pasos. Aprenderla aquí sirve para cualquier modelo del curso: árboles, SVM, redes neuronales...

Paso	Código	Qué hace
1 · Crear	<code>modelo = LinearRegression()</code>	Instancia el modelo con sus hiperparámetros.
2 · Entrenar	<code>modelo.fit(X, y)</code>	Aprende los parámetros a partir de los datos.
3 · Predecir	<code>modelo.predict(X_nuevo)</code>	Aplica el modelo aprendido a entradas nuevas.

7.1 Preparar los datos

```
In [4]: from sklearn.linear_model import LinearRegression
        from sklearn.model_selection import train_test_split
        from sklearn.metrics import (mean_squared_error, mean_absolute_error,
                                     r2_score)

        X = horas.reshape(-1, 1) # matriz 2D: (muestras, características)
        y = nota                  # el objetivo es un vector 1D

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=0)
        print('Entrenamiento:', X_train.shape, ' Prueba:', X_test.shape)
```

```
Out[4]: Entrenamiento: (48, 1)  Prueba: (12, 1)
```

NOTA ¿Por qué reshape(-1, 1) y por qué dividir los datos?

`reshape(-1, 1)` convierte el vector de 60 valores en una matriz de 60 filas y 1 columna; el `-1` le pide a NumPy que calcule ese número solo. scikit-learn siempre espera las características como matriz 2D, aunque haya una sola.

Separamos un 20 % de los datos como **conjunto de prueba**: el modelo no lo ve durante el entrenamiento, así que su error ahí mide qué tan bien *generaliza* a datos nuevos. Con 60 muestras quedan 48 para entrenar y 12 para probar.

7.2 Entrenar el modelo

```
In [5]: modelo = LinearRegression()
        modelo.fit(X_train, y_train)

        w_sk = modelo.coef_[0] # coef_: un peso por característica
        b_sk = modelo.intercept_ # intercept_ es un número

        print(f'Pendiente w = {w_sk:.4f} (valor real: 5)')
        print(f'Intercepto b = {b_sk:.4f} (valor real: 30)')
        print(f'\nModelo aprendido: nota = {b_sk:.2f} + {w_sk:.2f} · horas')
```

```
Out[5]: Pendiente w = 5.0675 (valor real: 5)
Intercepto b = 28.5394 (valor real: 30)

Modelo aprendido: nota = 28.54 + 5.07 · horas
```

Los parámetros no coinciden exactamente con 5 y 30 porque el modelo solo ve 48 muestras con ruido; aun así, se acercan bastante.

EJEMPLO Cómo se lee el modelo

Pendiente: cada hora adicional de estudio se asocia con unos 5,07 puntos más en la nota.

Intercepto: un estudiante que no estudia nada obtendría, según el modelo, unos 28,54 puntos.

Observa que el guion bajo final en `coef_` e `intercept_` es una convención de scikit-learn: marca los atributos que **se aprenden** durante `fit`.

7.3 Predecir y evaluar

```
In [6]: y_pred = modelo.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
print(f'MAE (error absoluto medio) = {mae:.3f} puntos')
print(f'MSE (error cuadrático medio) = {mse:.3f}')
print(f'RMSE (raíz del MSE) = {np.sqrt(mse):.3f} puntos')
print(f'R² (coef. de determinación) = {r2_score(y_test, y_pred):.3f}')
```

```
Out[6]: MAE (error absoluto medio) = 2.781 puntos
MSE (error cuadrático medio) = 12.188
RMSE (raíz del MSE) = 3.491 puntos
R² (coef. de determinación) = 0.946
```

DEFINICIÓN Cómo interpretar estos números

MAE = 2,78: en promedio, la predicción se equivoca en unos 2,8 puntos de nota.

RMSE = 3,49: similar, pero penaliza más los errores grandes; por eso siempre es $\geq$ MAE.

R² = 0,946: el modelo explica cerca del 95 % de la variabilidad de las notas del conjunto de prueba.

El RMSE debería quedar del orden del ruido que pusimos en los datos ($\sigma = 5$), que **ningún modelo puede eliminar**. Aquí sale algo menor por azar, porque el conjunto de prueba tiene solo 12 muestras.

7.4 Predecir para estudiantes nuevos

```
In [7]: nuevos = np.array([[2], [5], [8]])
for h, p in zip(nuevos.ravel(), modelo.predict(nuevos)):
    print(f'Con {h} horas de estudio se predice una nota de {p:.1f}')
```

```
Out[7]: Con 2 horas de estudio se predice una nota de 38.7
Con 5 horas de estudio se predice una nota de 53.9
Con 8 horas de estudio se predice una nota de 69.1
```

PRUEBA TÚ Verifícalo a mano

Sustituye $x = 5$ en el modelo aprendido: $28,54 + 5,07 \cdot 5 = 53,89 \approx 53,9$. `predict` no hace nada más misterioso que evaluar la ecuación (1) con los parámetros aprendidos.

7.5 Visualizar la recta ajustada

```
In [8]: xs = np.linspace(0, 10, 100).reshape(-1, 1)
plt.scatter(X_train, y_train, color=INDIGO, alpha=0.7, label='entrenamiento')
plt.scatter(X_test, y_test, color=CORAL, alpha=0.9, label='prueba')
plt.plot(xs, modelo.predict(xs), color=PURPURA, lw=2.5,
         label=f' $\hat{y} = \{w_{sk}:.2f\}x + \{b_{sk}:.2f\}$ ')
plt.xlabel('Horas de estudio'); plt.ylabel('Nota')
plt.title('Parte A: recta ajustada con scikit-learn'); plt.legend()
plt.show()
```

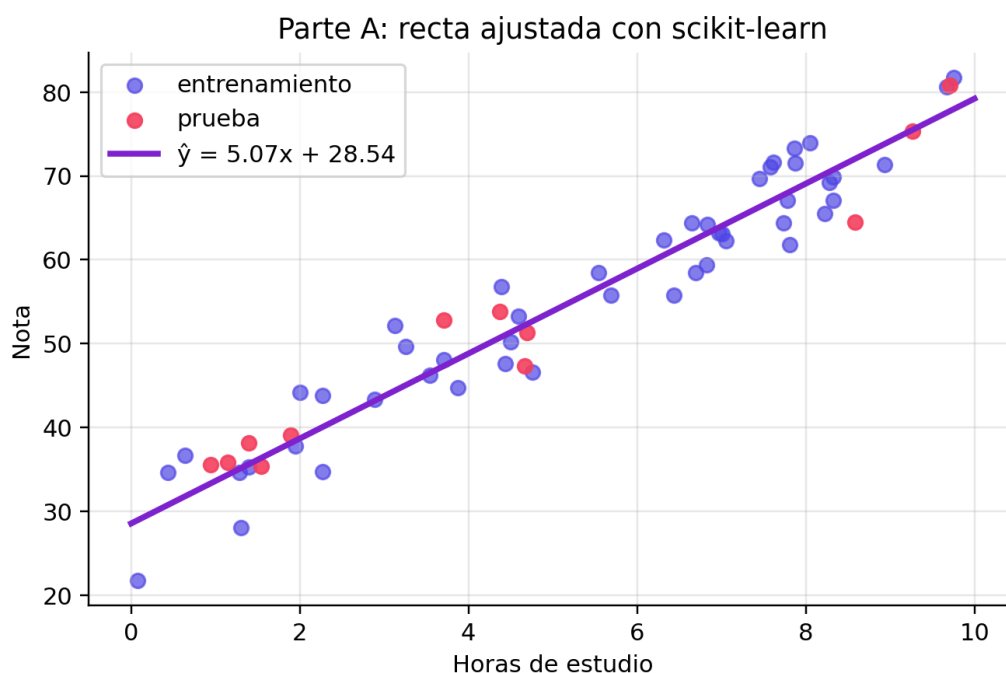


Figura 2. Recta ajustada por LinearRegression; en coral, los 12 puntos de prueba.

NOTA Lo que scikit-learn oculta

`LinearRegression` no itera: resuelve el problema de mínimos cuadrados de forma **directa**, con álgebra lineal (ecuación 3). Es rápido y exacto, pero oculta el proceso. En la Parte B lo construimos nosotros.

08 Parte B — Descenso de gradiente desde cero

Ahora olvidamos scikit-learn y construimos el algoritmo pieza por pieza con NumPy: la función de costo, el gradiente, la regla de actualización y el ciclo de entrenamiento.

8.1 La función de costo

Programamos directamente la ecuación (2). Gracias a la vectorización de NumPy no necesitamos ningún bucle: `w * x + b` calcula las 48 predicciones de una vez.

```
In [9]: def costo(w, b, x, y):
        """J(w, b) = (1/2m) Σ (w·x + b - y)²"""
        y_pred = w * x + b
        return np.mean((y_pred - y) ** 2) / 2

        x_train, x_test = X_train.ravel(), X_test.ravel() # aquí bastan vectores 1D

        print(f'J(0, 0) = {costo(0, 0, x_train, y_train):8.2f} <- recta en 0')
        print(f'J(5, 30) = {costo(5, 30, x_train, y_train):8.2f} <- relación real')
```

```
Out[9]: J(0, 0) = 1667.25 <- recta en 0
        J(5, 30) = 7.88 <- relación real
```

La recta $\hat{y} = 0$ es pésima ($J \approx 1667$), mientras que la relación real tiene un costo muy bajo ($J \approx 7,9$). El descenso de gradiente deberá llevarnos de la primera situación a la segunda.

8.2 El gradiente

El **gradiente** reúne las derivadas parciales de J : indica en qué dirección, y con qué intensidad, crece el costo al mover cada parámetro. Derivemos $\partial J / \partial w$ paso a paso con la regla de la cadena:

1. Llamamos $e^{(i)} = w \cdot x^{(i)} + b - y^{(i)}$ al residuo de la muestra i . Entonces $J = (1/2m) \sum (e^{(i)})^2$.
2. La derivada de $(e^{(i)})^2$ respecto a $e^{(i)}$ es $2 \cdot e^{(i)}$.
3. La derivada de $e^{(i)}$ respecto a w es $x^{(i)}$ (y respecto a b es 1).
4. Por la regla de la cadena: $\partial J / \partial w = (1/2m) \sum 2 \cdot e^{(i)} \cdot x^{(i)}$. El 2 se cancela con el $1/2$.

$$\begin{aligned}\partial J / \partial w &= (1/m) \cdot \sum (\hat{y}^{(i)} - y^{(i)}) \cdot x^{(i)} \\ \partial J / \partial b &= (1/m) \cdot \sum (\hat{y}^{(i)} - y^{(i)})\end{aligned}\tag{6}$$

IDEA CLAVE ¿Por qué $x^{(i)}$ aparece solo en $\partial J / \partial w$?

Porque w multiplica a x en el modelo, mientras que b va solo. En consecuencia, el gradiente de w queda «amplificado» por los valores de x (aquí, hasta 10). Este detalle explicará más adelante el comportamiento de la curva de aprendizaje.

```
In [10]: def gradientes(w, b, x, y):
          """Devuelve ( $\partial J/\partial w$ ,  $\partial J/\partial b$ )."""
          error = (w * x + b) - y          # residuo de cada muestra
          dw = np.mean(error * x)
          db = np.mean(error)
          return dw, db

          dw, db = gradientes(0, 0, x_train, y_train)
          print(f'En (w, b) = (0, 0):  $\partial J/\partial w$  = {dw:.2f},  $\partial J/\partial b$  = {db:.2f}')
```

```
Out[10]: En (w, b) = (0, 0):  $\partial J/\partial w$  = -339.86,  $\partial J/\partial b$  = -55.98
```

Ambos gradientes son **negativos**: aumentar w o b haría bajar el costo. Tiene sentido, porque la recta $\hat{y} = 0$ predice muy por debajo de las notas reales. Observa además que $|\partial J/\partial w|$ es unas seis veces mayor que $|\partial J/\partial b|$: es el efecto de multiplicar por x .

8.3 La regla de actualización

El descenso de gradiente da pasos en la dirección **opuesta** al gradiente (cuesta abajo), con un tamaño controlado por la tasa de aprendizaje α (ecuación 4). Si el gradiente es negativo, restar α veces un número negativo **aumenta** el parámetro, justo lo que queremos.

ADVERTENCIA Actualización simultánea

Los dos gradientes deben calcularse con los valores **actuales** de w y b , y solo después se actualizan ambos. Si actualizaras w primero y luego calcularas $\partial J/\partial b$ con el w nuevo, estarías implementando otro algoritmo. Nuestra función `gradientes` calcula dw y db juntos antes de modificar nada, así que lo cumple de forma natural.

8.4 El algoritmo completo

```
In [11]: def descenso_gradiente(x, y, alpha=0.01, iteraciones=10_000):
          w, b = 0.0, 0.0          # punto de partida
          historial = []          # J de cada iteración

          for i in range(iteraciones):
              dw, db = gradientes(w, b, x, y)          # 1) calcular el gradiente
              w = w - alpha * dw          # 2) dar un paso cuesta abajo
              b = b - alpha * db
              historial.append(costo(w, b, x, y)) # 3) registrar el costo

              if i in (0, 10, 100, 1000, iteraciones - 1):
                  print(f'iter {i:>5}: w = {w:7.4f} b = {b:8.4f}'
                        f'      J = {historial[-1]:9.4f}')

          return w, b, historial

          w_gd, b_gd, historial = descenso_gradiente(x_train, y_train,
                                                    alpha=0.01, iteraciones=10_000)
```

```
Out[11]: iter    0: w =  3.3986  b =   0.5598  J =  702.4903
iter   10: w =  8.9765  b =   1.9226  J =   77.3678
iter  100: w =  8.3954  b =   6.1889  J =   56.6861
iter 1000: w =  5.6493  b =  24.6320  J =    8.7716
iter 9999: w =  5.0675  b =  28.5394  J =    7.2610
```

En las primeras iteraciones el costo cae muchísimo y los parámetros cambian rápido; después los cambios son cada vez más finos, hasta que w y b se estabilizan en los mismos valores que obtuvo scikit-learn.

NOTA El sobrepaso de w

Hay un detalle curioso: w **se pasa** de su valor final (llega a casi 9 en la iteración 10) y luego retrocede hacia 5. Ocurre porque el gradiente respecto a w es mucho mayor que respecto a b , así que al principio casi todo el ajuste lo hace w . Como b todavía es pequeño, w «compensa» inclinando de más la recta para acercarse a los puntos; a medida que b crece, w vuelve a su lugar.

8.5 La curva de aprendizaje

```
In [12]: plt.plot(historial, color=INDIGO, lw=2)
plt.xscale('log'); plt.yscale('log')
plt.xlabel('iteración (escala log)'); plt.ylabel('J(w, b) (escala log)')
plt.title('Curva de aprendizaje')
plt.show()
```

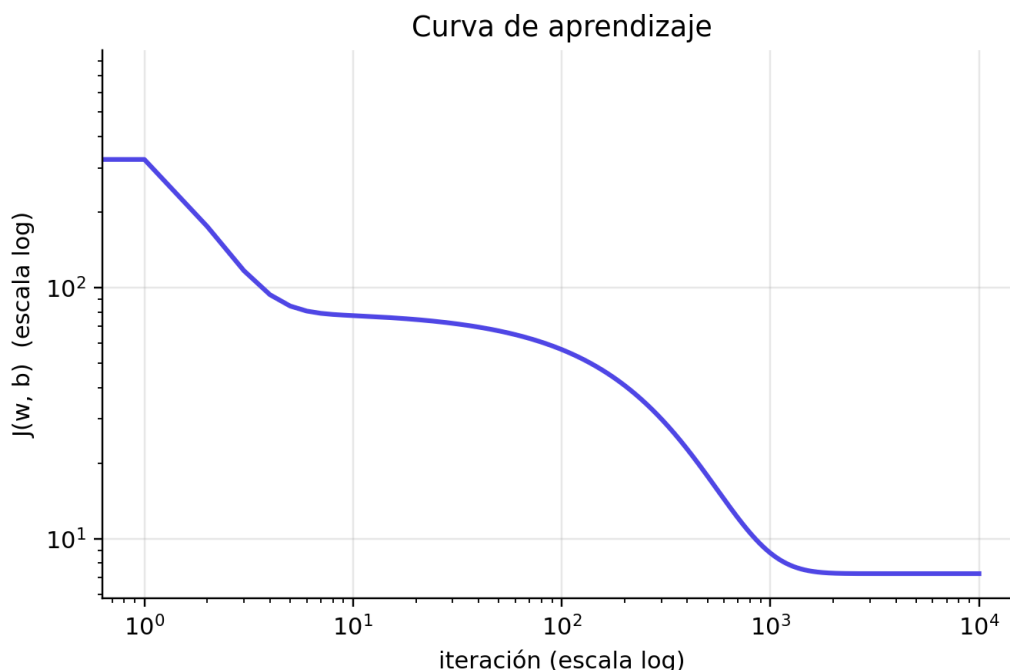


Figura 3. Curva de aprendizaje en escala log-log: caída inicial, meseta intermedia y convergencia.

La **curva de aprendizaje** es la herramienta principal para diagnosticar el entrenamiento. Debe **bajar y luego aplanarse**: la meseta final indica que llegamos al mínimo y que más iteraciones ya no aportan.

Aquí se distinguen dos fases. Entre las iteraciones 10 y 100 aparece una **meseta intermedia**: w ya se ajustó rápido, pero b avanza muy despacio porque su gradiente es pequeño. Recién hacia la iteración 1000 b alcanza su valor y el costo cae a su mínimo. Esa lentitud es el síntoma típico de **variables sin escalar**.

IDEA CLAVE Elegir la tasa de aprendizaje α

Con α muy pequeño la curva baja lentísimo; con α demasiado grande el costo **crece** en vez de bajar y el algoritmo diverge. Para estos datos el límite está cerca de $\alpha \approx 0,05$.

Escalar x (restar la media y dividir por la desviación estándar) iguala la magnitud de ambos gradientes, permite usar α mucho mayores y converger en muy pocas iteraciones. Lo comprobarás en los ejercicios 1 y 2.

8.6 Evaluar el modelo propio

Usamos los parámetros aprendidos para predecir sobre el conjunto de prueba y calculamos las mismas métricas de la Parte A, esta vez programándolas a mano a partir de las fórmulas de la sección 5.4.

```
In [13]: y_pred_gd = w_gd * x_test + b_gd

mae = np.mean(np.abs(y_test - y_pred_gd))
rmse = np.sqrt(np.mean((y_test - y_pred_gd) ** 2))
r2 = 1 - (np.sum((y_test - y_pred_gd) ** 2)
          / np.sum((y_test - y_test.mean()) ** 2))

print(f'MAE = {mae:.3f} puntos')
print(f'RMSE = {rmse:.3f} puntos')
print(f'R² = {r2:.3f}')
```

```
Out[13]: MAE = 2.781 puntos
RMSE = 3.491 puntos
R² = 0.946
```

DEFINICIÓN R^2 a mano

$R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}}$, donde SS_{res} es la suma de residuos al cuadrado del modelo y SS_{tot} la de un modelo que predice siempre la media. Si nuestro modelo no mejora a la media, $R^2 = 0$; si no comete errores, $R^2 = 1$. Puede incluso ser negativo si el modelo es peor que la media.

09 Comparación de ambos caminos

```
In [14]: print(f'{"":22}{w":>10}{b":>10}')
print(f'{"Valor real":22}{5:>10.4f}{30:>10.4f}')
print(f'{"scikit-learn":22}{w_sk:>10.4f}{b_sk:>10.4f}')
print(f'{"Descenso de gradiente":22}{w_gd:>10.4f}{b_gd:>10.4f}')
print(f'\nDiferencia |w_sk - w_gd| = {abs(w_sk - w_gd):.2e}')
print(f'Diferencia |b_sk - b_gd| = {abs(b_sk - b_gd):.2e}')
```

Out[14]:

	w	b
Valor real	5.0000	30.0000
scikit-learn	5.0675	28.5394
Descenso de gradiente	5.0675	28.5394

Diferencia $|w_{sk} - w_{gd}| = 1.55e-08$
Diferencia $|b_{sk} - b_{gd}| = 1.04e-07$

In [15]:

```
plt.scatter(horas, nota, color=INDIGO, alpha=0.5, label='datos')
plt.plot(xs, modelo.predict(xs), color=PURPURA, lw=5, alpha=0.5,
        label='scikit-learn')
plt.plot(xs, w_gd * xs + b_gd, color=CORAL, lw=1.5, ls='--',
        label='descenso de gradiente')
plt.xlabel('Horas de estudio'); plt.ylabel('Nota')
plt.title('Ambos métodos producen la misma recta'); plt.legend()
plt.show()
```

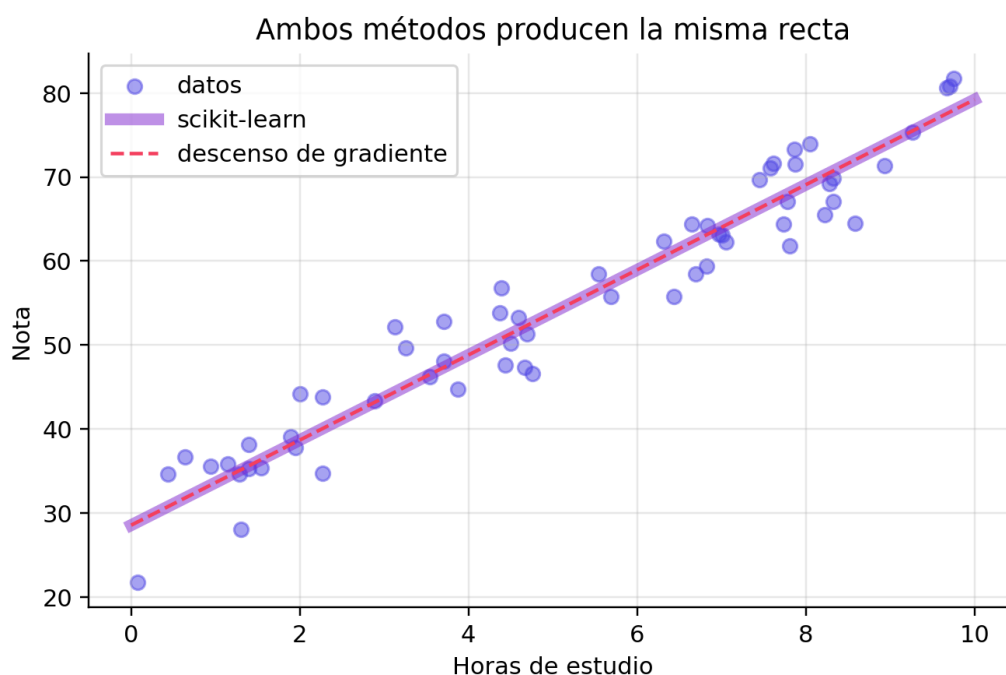


Figura 4. La recta de scikit-learn (púrpura) y la del descenso de gradiente (coral, discontinua) se superponen.

Las dos rectas se superponen: los parámetros coinciden hasta la séptima cifra decimal. La diferencia residual (del orden de 10^{-8}) se debe a que el descenso de gradiente se detuvo tras 10 000 iteraciones, a un paso infinitesimal del mínimo exacto.

Aspecto	scikit-learn	Descenso de gradiente
Cómo resuelve	Directamente (álgebra lineal)	Paso a paso, bajando por la función de costo
Resultado	Exacto	Aproximado; mejora con las iteraciones
Hiperparámetros	Ninguno	Tasa de aprendizaje α e iteraciones
Líneas de código	2	~15

Aspecto	scikit-learn	Descenso de gradiente
Valor para el curso	Herramienta de uso diario	Base de la regresión logística y de las redes neuronales

IDEA CLAVE Conclusión

Para una regresión lineal real, `LinearRegression` es la opción correcta: exacta y sin ajustes. Pero el descenso de gradiente es el motor que entrena casi todos los modelos que veremos después, porque la mayoría **no tiene** una solución directa. Entenderlo aquí, en el caso más simple, es la preparación para todo lo que sigue.

10 Entregables

- Enlace a su cuaderno Jupyter en GitHub **ejecutado de principio a fin** (Kernel → Restart & Run All), con el nombre `GL09_Apellido_Nombre.ipynb`.

11 Rúbrica de evaluación

Criterio	Descripción	Pts.
Parte A	Preparación de datos, entrenamiento, métricas y gráfico reproducidos correctamente	50
Parte B	Costo, gradientes, regla de actualización y ciclo implementados y funcionando; curva de aprendizaje	50
Total		100

13 Referencias

- [1] Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3.ª ed.), cap. 4: Training Models. O'Reilly.
- [2] James, G., Witten, D., Hastie, T., Tibshirani, R. y Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*, cap. 3: Linear Regression. Springer.
- [3] Ng, A. *Machine Learning Specialization*, curso 1: Supervised Machine Learning — Regression and Classification. DeepLearning.AI / Coursera.
- [4] scikit-learn developers. *LinearRegression* y *Metrics and scoring: regression metrics*. Documentación oficial de scikit-learn.

IDEA CLAVE Fin de la guía

Resolviste la regresión lineal por el camino directo y por el iterativo, y comprobaste que llegan al mismo lugar. En la próxima guía extenderemos estas ideas a la **regresión polinomial** y al problema de sobreajuste.